

TRACK: DATA SCIENCE

Completion Rule: Complete **at least 3** of the 5 tasks below.

TASK 1 • Iris Flower Classification

Objective: Train a machine learning classification model to identify the species of an iris flower (Setosa, Versicolor, or Virginica) from its physical measurements.

Tech Stack: Python, scikit-learn, pandas, matplotlib/seaborn, Jupyter Notebook

Feature Checklist:

- [] Load the Iris dataset (available directly from `sklearn.datasets.load_iris()` — no download required)
- [] Perform **Exploratory Data Analysis (EDA)**: shape, dtypes, null value check, descriptive statistics
- [] Visualisations: pairplot or scatter matrix showing feature distributions by species; box plots for each feature
- [] **Feature selection** discussion: which features are most discriminative?
- [] Train/test split (typically 80/20) using `train_test_split`
- [] Train at least **2 different classifiers** (e.g., Logistic Regression, K-Nearest Neighbours, Decision Tree, Random Forest)
- [] Evaluate each model: accuracy score, confusion matrix, classification report (precision, recall, F1)
- [] Identify and declare the best-performing model with justification
- [] All code in a clean, commented Jupyter Notebook

Self-Sourcing Guideline: The dataset is built into scikit-learn — no external download needed. Search "**Iris flower classification Python scikit-learn tutorial**" on YouTube for an end-to-end walkthrough. Reference the **scikit-learn User Guide** (scikit-learn.org) for model-specific documentation. For visualisations, search "**seaborn pairplot tutorial**".

TASK 2 • Unemployment Analysis with Python

Objective: Perform exploratory data analysis on unemployment data to uncover regional and temporal trends, with a focus on the impact of the COVID-19 pandemic on unemployment rates in India.

Tech Stack: Python, pandas, matplotlib, seaborn, Jupyter Notebook

Feature Checklist:

- [] Download a suitable dataset (see Self-Sourcing Guideline below)
- [] Data loading, shape inspection, null value check, and type conversion

- [] EDA: region-wise average unemployment rates, month-wise trends
- [] Time-series line chart: unemployment rate over time for at least 3 major states/regions
- [] Bar chart: top 10 states with highest average unemployment rate
- [] Heatmap: correlation between unemployment rate, employment rate, and labour participation rate
- [] Pre-COVID vs. post-COVID comparison (split data by date, calculate mean rates for each period)
- [] Written observations/markdown cells between each chart explaining what the data shows
- [] Clean, well-commented Jupyter Notebook

Self-Sourcing Guideline: Search "**unemployment rate India dataset**" on **Kaggle.com** (create a free account). The dataset titled "**Unemployment in India**" is publicly available there. Search "**pandas time series analysis tutorial**" and "**seaborn heatmap tutorial**" on YouTube for the core visualisation patterns.

TASK 3 · Car Price Prediction with Machine Learning

Objective: Build a regression model that predicts the selling price of a used car based on features such as brand, age, mileage, fuel type, and transmission.

Tech Stack: Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook

Feature Checklist:

- [] Download a suitable dataset (see Self-Sourcing Guideline below)
- [] Data cleaning: handle null values, remove duplicates, address inconsistent categorical values (e.g., "Petrol" vs. "petrol")
- [] Feature engineering: calculate **car age** from the year column; extract brand from the car name column
- [] EDA: distribution of selling prices, price vs. fuel type box plots, price vs. car age scatter plot
- [] Encode categorical variables (One-Hot Encoding or Label Encoding)
- [] Feature correlation heatmap
- [] Train/test split
- [] Train at least **2 regression models** (e.g., Linear Regression, Random Forest Regressor, Gradient Boosting)
- [] Evaluate models using: MAE, RMSE, and R^2 score
- [] Feature importance chart for your best-performing model
- [] All steps in a clean, commented Jupyter Notebook

Self-Sourcing Guideline: Search "**car price prediction dataset**" on **Kaggle.com**. A widely used dataset is "**Vehicle dataset from cardekho**" — it is publicly available and well-suited for this task. Search "**car price prediction Python machine learning tutorial**" on YouTube for guidance.

TASK 4 · Email Spam Detection with Machine Learning

Objective: Build a Natural Language Processing (NLP) binary classifier that distinguishes spam emails from legitimate (ham) emails.

Tech Stack: Python, pandas, scikit-learn (TF-IDF, Naive Bayes/SVM), NLTK or re, Jupyter Notebook

Feature Checklist:

- [] Download a suitable dataset (see Self-Sourcing Guideline below)
- [] Data loading and class distribution check (spam vs. ham counts and percentage)
- [] Text preprocessing pipeline: lowercase conversion, punctuation removal, stopword removal, optional stemming/lemmatization
- [] Feature extraction using **TF-IDF Vectorizer** (explain what TF-IDF measures in a markdown cell)
- [] Train/test split
- [] Train at least **2 classifiers**: Multinomial Naive Bayes (industry standard for text) + one alternative (e.g., Logistic Regression, SVM)
- [] Evaluation: accuracy, precision, recall, F1-score, confusion matrix
- [] Discussion: Why is **recall** particularly important for spam detection? (Answer in a markdown cell)
- [] (Bonus) WordCloud visualisations for spam words and ham words
- [] All steps in a clean, commented Jupyter Notebook

Self-Sourcing Guideline: Search "**SMS spam collection dataset**" on **Kaggle.com** or the **UCI Machine Learning Repository** (archive.ics.uci.edu) — both host this classic dataset for free. Search "**email spam detection Python NLP tutorial scikit-learn**" on YouTube. Reference the NLTK documentation (nltk.org) for text preprocessing.

TASK 5 · Sales Prediction Using Python

Objective: Build a regression model that predicts product sales based on advertising spend across different media channels (TV, Radio, Newspaper).

Tech Stack: Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook

Feature Checklist:

- [] Download a suitable dataset (see Self-Sourcing Guideline below)
- [] Data loading and EDA: null check, descriptive statistics, pairplot of all features
- [] Individual scatter plots: Sales vs. TV spend, Sales vs. Radio spend, Sales vs. Newspaper spend
- [] Correlation matrix heatmap

- [] Train/test split
- [] Train a **Linear Regression** model as the baseline
- [] Train at least one additional model (e.g., Random Forest Regressor, Polynomial Regression)
- [] Evaluate using: MAE, RMSE, R^2 score
- [] Residual plot for the best model (are errors randomly distributed or systematic?)
- [] Interpretation: which advertising channel has the highest impact on sales? (Answered using coefficients or feature importance)
- [] Clean, well-commented Jupyter Notebook

Self-Sourcing Guideline: Search "**advertising sales prediction dataset**" on **Kaggle.com**. The classic "**Advertising.csv**" dataset (TV, Radio, Newspaper → Sales) is widely available. Search "**sales prediction linear regression Python tutorial**" on YouTube. Reference the **scikit-learn documentation** for LinearRegression and evaluation metrics.

